

Info-SLAM — Consolidation développement

Cahier de vacances · BTS SIO

Matière : info-slam

Durée estimée : 10h

Objectifs

- Consolider les requêtes SQL SELECT, WHERE, ORDER BY
- Maîtriser les jointures SQL et les fonctions d'agrégation
- Comprendre et écrire une classe Python avec attributs et méthodes
- Appliquer l'héritage et le polymorphisme en POO
- Utiliser les structures de contrôle et les fonctions en Python
- Gérer les erreurs avec try/except en Python
- Utiliser Git pour versionner son code (commit, branche, merge)
- Réaliser un mini-projet CRUD complet (modèle, DAO, contrôleur, vue)

Sommaire

1. Bienvenue dans ton cahier Info-SLAM (15min)
2. SQL — SELECT, WHERE, ORDER BY (60min)
3. SQL — JOIN, GROUP BY et agrégats (75min)
4. POO — classe, attribut, méthode (75min)
5. POO — héritage et polymorphisme (75min)
6. Algorithmie — boucles, conditions, fonctions (75min)
7. Gestion d'erreurs et exceptions (60min)
8. Git — commit, branche, merge (60min)
9. Mini-projet CRUD — Gestion de contacts (90min)
10. Bilan final — Info-SLAM (20min)

Bienvenue dans ton cahier Info-SLAM

Bonjour !

Ce cahier de vacances est là pour t'aider à **consolider tes bases en développement** avant la 2ème année de BTS SIO SLAM. Il n'est pas un examen, il n'y a pas de note — c'est un outil pour toi, à ton rythme.

Si certains thèmes te semblent déjà familiers, tant mieux : les activités te permettront de vérifier que tu les maîtrises vraiment. Si d'autres te semblent flous, les rappels de cours et les aides IA sont là pour t'accompagner.

Ce que tu vas travailler

Ce cahier couvre **8 thèmes**, pour environ 10 heures de travail au total :

Activité	Thème	Durée
01	SQL — SELECT, WHERE, ORDER BY	60 min
02	SQL — JOIN, GROUP BY, agrégats	75 min
03	POO — classe, attribut, méthode	75 min
04	POO — héritage et polymorphisme	75 min
05	Algorithmie — boucles, conditions, fonctions	75 min
06	Gestion d'erreurs et exceptions	60 min
07	Git — commit, branche, merge	60 min
08	Mini-projet CRUD complet	90 min

Comment utiliser ce cahier

1. **Lis le rappel de cours** — quelques paragraphes, l'essentiel pour l'activité.
2. **Fais les activités** — lis du code, réponds aux QCM, écris des petites requêtes ou fonctions.
3. **Utilise l'aide IA si tu bloques** — chaque activité a un prompt tout prêt à copier dans ChatGPT ou Claude. Il te guidera **sans te donner la réponse directe**.
4. **Regarde la solution** après avoir essayé — elle t'explique le raisonnement.
5. **Fais ton auto-évaluation** — sois honnête avec toi-même, ça t'aide à savoir sur quoi revenir.

Un conseil avant de commencer

Ne cherche pas à tout faire d'un coup. **Une activité par jour** pendant les vacances, c'est amplement suffisant. Le but est de rentrer en septembre avec les fondamentaux solides.

Les activités 01 et 02 sont de la révision SQL — un bon point de départ accessible. Les activités 03 à 05 montent progressivement vers la POO et l'algorithmie. Les activités 06 et 07 consolident des compétences transversales. L'activité 08 est un mini-projet de synthèse.

Aide IA — apprendre avec l'intelligence artificielle

Chaque activité propose un **encart Aide IA** avec un prompt-type. L'idée : si tu bloques, tu colles ce prompt dans ChatGPT ou Claude, et tu demandes de l'aide à **comprendre**, pas à copier une réponse.

□ Aide IA — prompt à copier :

```
Je suis étudiant en BTS SIO option SLAM et je reprends mes révisions pendant les vacances. Je travaille sur [le thème de l'activité].  
Je bloque sur [explique ce que tu ne comprends pas].  
Aide-moi à comprendre pas à pas, sans me donner la réponse directe.  
Pose-moi des questions pour que je trouve moi-même.
```

Bonne révision !

SQL — SELECT, WHERE, ORDER BY

Objectifs :

- Lire et comprendre une requête SELECT simple
- Filtrer des résultats avec WHERE
- Trier les résultats avec ORDER BY

Rappel de cours

Une requête SELECT permet d'extraire des données d'une table.

```
-- Structure de base
SELECT colonne1, colonne2
FROM table
WHERE condition
ORDER BY colonne ASC;
-- Exemples concrets
SELECT nom, prenom FROM clients;
SELECT nom, prenom FROM clients WHERE ville = 'Lyon';
SELECT nom, prenom FROM clients WHERE ville = 'Lyon' ORDER BY nom ASC;
```

Quelques points clés :

- SELECT * sélectionne **toutes** les colonnes (pratique mais à éviter en production).
- WHERE filtre les lignes selon une condition. On peut combiner avec AND et OR.
- ORDER BY trie : ASC (croissant, défaut) ou DESC (décroissant).

Activité 1 — Lire une requête

Voici la table produits :

	id	nom	prix	stock
1	Clavier	45	12	
2	Souris	25	0	
3	Écran	299	5	
4	Webcam	89	3	

Et la requête suivante :

```
SELECT nom, prix
FROM produits
```

```
WHERE stock > 0  
ORDER BY prix DESC;
```

Q : Quels produits cette requête renvoie-t-elle, dans quel ordre ?

- ☐ Tous les produits, triés du moins cher au plus cher
- ☐ Les produits en stock (stock > 0), triés du plus cher au moins cher
- ☐ Les produits en stock (stock > 0), triés du moins cher au plus cher
- ☐ Tous les produits, triés du plus cher au moins cher

Q : La requête renvoie seulement les colonnes nom et prix. Si on remplace "SELECT nom, prix" par "SELECT *", que se passe-t-il ?

- ☐ Rien ne change, c'est pareil
- ☐ {'La requête renvoie toutes les colonnes': 'id, nom, prix, stock '}
- ☐ La requête renvoie une erreur
- ☐ La requête renvoie uniquement la colonne id

Activité 2 — Compléter une requête

Tu veux la liste des produits dont le prix est inférieur à 100€, triés par nom alphabétique.

Q : Écris la clause WHERE manquante (juste la condition, sans le mot WHERE).

Q : Écris la clause ORDER BY complète pour trier par nom alphabétique.

Activité 3 — Écrire une requête complète

La table clients a les colonnes : id, nom, prenom, ville, age.

Tu veux afficher le prénom et le nom de tous les clients de Paris, triés par âge décroissant.

Q : Écris la requête SQL complète (les espaces et la casse seront ignorés).

Aide IA

📄 **Aide IA — prompt à copier :**

```
Je suis étudiant en BTS SIO option SLAM et je révise SQL.  
Je dois écrire une requête SELECT avec WHERE et ORDER BY mais j'ai du mal  
à me souvenir de l'ordre des clauses et de la syntaxe exacte.  
Pose-moi des questions pour que je reconstruise la requête moi-même,  
sans me donner la réponse directe.
```

Solution

Voir les réponses (à la fin du cahier)

Auto-évaluation

Où en es-tu ? ☐ Compris ☐ À revoir ☐ Bloqué

SQL — JOIN, GROUP BY et agrégats

Objectifs :

- Joindre deux tables avec INNER JOIN
- Regrouper des résultats avec GROUP BY
- Utiliser les fonctions COUNT, SUM, AVG, MAX, MIN

Rappel de cours

Joindre deux tables

Quand des données sont réparties sur plusieurs tables, on utilise JOIN pour les combiner :

```
SELECT commandes.id, clients.nom, commandes.montant
FROM commandes
INNER JOIN clients ON clients.id = commandes.id_client;
```

INNER JOIN ne renvoie que les lignes qui ont une correspondance dans les deux tables. On peut utiliser des **alias** pour raccourcir :

```
SELECT c.id, cl.nom, c.montant
FROM commandes c
INNER JOIN clients cl ON cl.id = c.id_client;
```

Regrouper et compter

GROUP BY regroupe les lignes qui ont la même valeur dans une colonne. On l'utilise avec des **fonctions d'agrégation** :

Fonction Rôle

COUNT(*) Compte le nombre de lignes

SUM(col) Somme les valeurs

AVG(col) Calcule la moyenne

MAX(col) Valeur maximale

MIN(col) Valeur minimale

```
-- Nombre de commandes par client
SELECT id_client, COUNT(*) AS nb_commandes
FROM commandes
GROUP BY id_client;
-- Montant total des commandes par client
```

```
SELECT id_client, SUM(montant) AS total
FROM commandes
GROUP BY id_client;
```

Activité 1 — Lire un JOIN

Voici deux tables :

clients : id, nom, ville **commandes** : id, id_client, produit, montant

```
SELECT cl.nom, c.produit, c.montant
FROM commandes c
INNER JOIN clients cl ON cl.id = c.id_client
WHERE cl.ville = 'Bordeaux'
ORDER BY c.montant DESC;
```

Q : Que renvoie cette requête ?

- ☐ Le nom et la ville de tous les clients
- ☐ Le nom du client, le produit commandé et le montant, uniquement pour les clients de Bordeaux, triés du montant le plus élevé au plus faible
- ☐ Toutes les commandes de tous les clients, triées par montant
- ☐ Le nombre de commandes des clients de Bordeaux

Q : Dans "ON cl.id = c.id_client", à quoi sert cette condition ?

- ☐ Elle filtre les lignes comme un WHERE
- ☐ {'Elle définit le lien entre les deux tables': "une commande appartient au client dont l'id correspond "}
- ☐ Elle trie les résultats
- ☐ Elle regroupe les résultats par client

Activité 2 — Lire un GROUP BY

```
SELECT cl.nom, COUNT(*) AS nb_commandes, SUM(c.montant) AS total
FROM commandes c
INNER JOIN clients cl ON cl.id = c.id_client
GROUP BY cl.nom
ORDER BY total DESC;
```

Q : Que représente chaque ligne du résultat de cette requête ?

- ☐ Une commande individuelle avec le nom du client
- ☐ Un client, avec le nombre total de ses commandes et le montant total qu'il a dépensé
- ☐ Un produit commandé, avec le total des ventes
- ☐ Un résumé global de toutes les commandes

Activité 3 — Écrire une requête avec JOIN

Voici les tables : - produits(id, nom, prix, id_categorie) - categories(id, nom)

Écris une requête qui renvoie le nom du produit et le nom de sa catégorie, pour tous les produits dont le prix est supérieur à 50€.

Q : Écris la requête SQL complète (les espaces et la casse seront ignorés).

Activité 4 — Écrire un GROUP BY

Avec la même table produits, écris une requête qui affiche le prix moyen des produits pour chaque catégorie (id_categorie) .

Q : Complète la requête — quelle clause vient après SELECT id_categorie, AVG(prix) AS prix_moyen FROM produits ?

Aide IA

☐ **Aide IA — prompt à copier :**

Je suis étudiant en BTS SIO option SLAM et je révise SQL.
Je dois écrire une requête avec INNER JOIN et GROUP BY mais je ne comprends pas bien comment articuler ces deux clauses ensemble.

Explique-moi par étapes, avec un exemple différent de celui de mon cours, sans me donner directement la réponse à mon exercice.

Solution

Voir les réponses (à la fin du cahier)

Auto-évaluation

Où en es-tu ? ☐ Compris ☐ À revoir ☐ Bloqué

POO — classe, attribut, méthode

Objectifs :

- Comprendre ce qu'est une classe et un objet
- Lire et écrire une classe Python avec `__init__`
- Appeler une méthode sur un objet

Rappel de cours

En **programmation orientée objet (POO)**, on modélise le monde réel avec des **classes**. Une classe est un moule ; un **objet** est une instance de ce moule.

```
class Produit:
    """Représente un produit dans un catalogue."""
    def __init__(self, nom, prix):
        self.nom = nom        # attribut
        self.prix = prix      # attribut
    def afficher(self):
        """Affiche les infos du produit."""
        print(f"{self.nom} – {self.prix} €")
    def appliquer_remise(self, pourcentage):
        """Applique une remise en % et retourne le nouveau prix."""
        return self.prix * (1 - pourcentage / 100)

# Créer des objets (instances)
clavier = Produit("Clavier", 45)
ecran = Produit("Écran", 299)
# Appeler des méthodes
clavier.afficher()                # Clavier – 45 €
print(ecran.appliquer_remise(10)) # 269.1
```

Points clés : - `__init__` est le **constructeur** : il est appelé quand on crée un objet. - `self` désigne l'objet lui-même. Il faut l'écrire en premier paramètre de chaque méthode. - Un **attribut** stocke une donnée (`self.nom`). Une **méthode** réalise une action.

Activité 1 — Lire une classe

```
class Rectangle:
    def __init__(self, largeur, hauteur):
        self.largeur = largeur
        self.hauteur = hauteur
    def aire(self):
        return self.largeur * self.hauteur
    def perimetre(self):
```

```
        return 2 * (self.largeur + self.hauteur)
r = Rectangle(5, 3)
print(r.aire())
print(r.perimetre())
```

Q : Que va afficher `print(r.aire())` ?

- ☐ 8
- ☐ 15
- ☐ 16
- ☐ 30

Q : Que va afficher `print(r.perimetre())` ?

- ☐ 8
- ☐ 15
- ☐ 16
- ☐ 30

Q : Dans la méthode `aire`, pourquoi écrit-on `"self.largeur"` plutôt que juste `"largeur"` ?

- ☐ C'est une convention sans utilité réelle
- ☐ Parce que `"self.largeur"` accède à l'attribut de l'objet, stocké lors du `__init__`
- ☐ Parce que Python l'exige pour les variables locales
- ☐ Pour éviter de confondre avec une variable globale

Activité 2 — Compléter une classe

```
class Eleve:
    def __init__(self, nom, age):
        self.nom = nom
        self.age = age
        self.notes = []    # liste vide au départ
    def ajouter_note(self, note):
        self.notes.append(note)
    def moyenne(self):
        # À compléter : retourne la moyenne des notes
```

```
# ou 0 si aucune note  
pass
```

Q : Complète la méthode `moyenne()`. Que doit retourner cette méthode si `self.notes` est vide ?

Q : Voici une tentative de complétion de `moyenne()` : `if len(self.notes) == 0:`
`return 0` `return sum(self.notes) / len(self.notes)` Cette implémentation est-elle
correcte ?

- ☐ Non, on ne peut pas utiliser `sum()` sur une liste de notes
- ☐ Oui, elle gère bien le cas vide et calcule correctement la moyenne
- ☐ Non, il faut utiliser une boucle `for` au lieu de `sum()`
- ☐ Non, `len()` ne fonctionne pas sur une liste

Activité 3 — Écrire une classe

Écris une classe `CompteBancaire` qui : - a un attribut `solde` initialisé à 0 dans `__init__` - a une méthode `deposer(montant)` qui ajoute le montant au solde - a une méthode `retirer(montant)` qui retire le montant du solde

Q : Comment s'appelle le constructeur en Python ?

Q : Dans la méthode `deposer(self, montant)`, comment ajouter montant au solde ?

Aide IA

📄 Aide IA — prompt à copier :

Je suis étudiant en BTS SIO SLAM et j'apprends la POO en Python.
Je dois écrire une classe avec un constructeur `__init__` et des méthodes,
mais je ne comprends pas bien le rôle de "self".
Explique-moi ce que représente "self" avec un exemple simple,
sans me donner le code de mon exercice directement.

Solution

Voir la solution (à la fin du cahier)

Auto-évaluation

Où en es-tu ? ☐ Compris ☐ À revoir ☐ Bloqué

POO — héritage et polymorphisme

Objectifs :

- Comprendre ce qu'est l'héritage en POO
- Lire une classe qui hérite d'une autre
- Identifier le polymorphisme dans du code Python

Rappel de cours

Héritage

L'**héritage** permet de créer une classe enfant qui réutilise les attributs et méthodes d'une classe parente, et peut en ajouter ou en modifier.

```
class Animal:
    def __init__(self, nom):
        self.nom = nom
    def parler(self):
        return "..."
    def presentation(self):
        return f"Je m'appelle {self.nom} et je dis : {self.parler()}"

class Chien(Animal):    # Chien hérite de Animal
    def parler(self):
        return "Woof !"

class Chat(Animal):    # Chat hérite de Animal
    def parler(self):
        return "Miaou !"

rex = Chien("Rex")
mimi = Chat("Mimi")
print(rex.presentation())    # Je m'appelle Rex et je dis : Woof !
print(mimi.presentation())    # Je m'appelle Mimi et je dis : Miaou !
```

Polymorphisme

Le **polymorphisme** signifie que des objets de types différents répondent à la même méthode, chacun à sa façon. Dans l'exemple ci-dessus, `parler()` est appelée sur des objets différents et retourne des valeurs différentes.

super()

`super()` appelle la méthode de la classe parente :

```
class Employe:
    def __init__(self, nom, salaire):
        self.nom = nom
        self.salaire = salaire
class Manager(Employe):
    def __init__(self, nom, salaire, equipe):
        super().__init__(nom, salaire)    # appelle Employe.__init__
        self.equipe = equipe              # attribut supplémentaire
```

Activité 1 — Lire du code avec héritage

```
class Vehicule:
    def __init__(self, marque, vitesse_max):
        self.marque = marque
        self.vitesse_max = vitesse_max
    def description(self):
        return f"{self.marque}, max {self.vitesse_max} km/h"

class Voiture(Vehicule):
    def __init__(self, marque, vitesse_max, nbportes):
        super().__init__(marque, vitesse_max)
        self.nbportes = nbportes
    def description(self):
        base = super().description()
        return f"{base}, {self.nbportes} portes"

v = Voiture("Renault", 180, 5)
print(v.description())
```

Q : Que va afficher `print(v.description())` ?

- ☐ Renault, max 180 km/h
- ☐ Renault, max 180 km/h, 5 portes
- ☐ Voiture, max 180 km/h, 5 portes
- ☐ Une erreur, car `description()` est définie deux fois

Q : À quoi sert `super().__init__(marque, vitesse_max)` dans `Voiture.__init__` ?

- ☐ À créer un nouvel objet `Vehicule` séparé
- ☐ À appeler le constructeur de la classe parente `Vehicule` pour initialiser `marque` et `vitesse_max`
- ☐ À copier tous les attributs de `Vehicule`
- ☐ C'est obligatoire en Python, même si ça ne fait rien

Activité 2 — Polymorphisme en action

```
class Forme:
    def aire(self):
        return 0
class Carre(Forme):
    def __init__(self, cote):
        self.cote = cote
    def aire(self):
        return self.cote ** 2
class Cercle(Forme):
    def __init__(self, rayon):
        self.rayon = rayon
    def aire(self):
        return 3.14 * self.rayon ** 2
formes = [Carre(4), Cercle(3), Carre(2)]
for f in formes:
    print(f.aire())
```

Q : Quelles valeurs affiche ce programme (dans l'ordre) ?

- ☐ 0, 0, 0
- ☐ 4, 3, 2
- ☐ 16, 28.26, 4
- ☐ Une erreur car Forme.aire() retourne 0

Q : Pourquoi peut-on appeler f.aire() sur des objets de types Carre et Cercle dans la même boucle ?

- ☐ Parce que Python convertit automatiquement les types
- ☐ Parce que les deux classes héritent de Forme et définissent toutes les deux une méthode aire() — c'est le polymorphisme
- ☐ Parce que Carre et Cercle sont des synonymes en Python
- ☐ Parce que la liste formes ne peut contenir que des objets du même type

Activité 3 — Compléter une classe fille

```
class Personne:
    def __init__(self, nom, age):
        self.nom = nom
        self.age = age
    def presenter(self):
```

```
return f"Je suis {self.nom}, j'ai {self.age} ans."
```

```
class Etudiant(Personne):  
    def __init__(self, nom, age, formation):  
        # À compléter : appeler le constructeur parent  
        self.formation = formation  
    def presenter(self):  
        # À compléter : utiliser super() et ajouter la formation  
        pass
```

Q : Dans `Etudiant.__init__`, quelle ligne appelle le constructeur de `Personne` ?

Q : Dans `Etudiant.presenter()`, que retourne `super().presenter()` ?

Aide IA

📄 **Aide IA — prompt à copier :**

```
Je suis étudiant en BTS SIO SLAM et j'étudie l'héritage en POO Python.  
Je ne comprends pas bien comment super() fonctionne et pourquoi on  
l'utilise dans __init__.  
Explique-moi avec un exemple très simple, sans me donner le code  
de mon exercice. Pose-moi des questions pour tester ma compréhension.
```

Solution

Voir la solution (à la fin du cahier)

Auto-évaluation

Où en es-tu ? ☐ Compris ☐ À revoir ☐ Bloqué

Algorithmie — boucles, conditions, fonctions

Objectifs :

- Lire et écrire des boucles for et while en Python
- Utiliser les conditions if/elif/else
- Définir et appeler une fonction avec paramètres et valeur de retour

Rappel de cours

Conditions

```
note = 12
if note >= 16:
    mention = "Très bien"
elif note >= 14:
    mention = "Bien"
elif note >= 12:
    mention = "Assez bien"
elif note >= 10:
    mention = "Passable"
else:
    mention = "Insuffisant"
print(mention)    # Assez bien
```

Boucles

```
# Boucle for : parcourir une liste ou une plage
fruits = ["pomme", "banane", "cerise"]
for fruit in fruits:
    print(fruit)
# range(n) : de 0 à n-1
for i in range(5):
    print(i)          # 0, 1, 2, 3, 4
# Boucle while : tant que la condition est vraie
compteur = 0
while compteur < 3:
    print(compteur)
    compteur += 1     # 0, 1, 2
```

Fonctions

```
def calculer_tva(prix_ht, taux=20):
    """Retourne le prix TTC."""
    return prix_ht * (1 + taux / 100)
```

```
prix_ttc = calculer_tva(100)          # 120.0 (taux par défaut = 20%)
prix_ttc2 = calculer_tva(100, 10)    # 110.0 (taux = 10%)
```

Activité 1 — Lire du code avec boucle et condition

```
notes = [8, 15, 12, 6, 18, 10]
bonnes_notes = 0
for note in notes:
    if note >= 10:
        bonnes_notes += 1
print(f"Notes >= 10 : {bonnes_notes}")
```

Q : Que va afficher ce programme ?

- ☐ {'Notes >= 10': 3}
- ☐ {'Notes >= 10': '4 '}
- ☐ {'Notes >= 10': 5}
- ☐ {'Notes >= 10': 6}

Q : Si on remplace "if note >= 10" par "if note > 10", combien de bonnes notes compte-t-on ?

- ☐ 2
- ☐ 3
- ☐ 4
- ☐ 5

Activité 2 — Lire une fonction

```
def est_pair(n):
    return n % 2 == 0
def compter_pairs(liste):
    compteur = 0
    for element in liste:
        if est_pair(element):
            compteur += 1
    return compteur
nombres = [1, 2, 3, 4, 5, 6]
print(compter_pairs(nombres))
```

Q : Que va afficher `print(compter_paires(nombres))` ?

- ☐ 2
- ☐ 3
- ☐ 4
- ☐ 6

Q : Que retourne `est_pair(7)` ?

- ☐ 7
- ☐ True
- ☐ False
- ☐ 1

Activité 3 — Compléter une fonction

```
def maximum(liste):  
    """Retourne le plus grand élément d'une liste non vide."""  
    maxi = liste[0]  
    for element in liste:  
        if element > maxi:  
            maxi = element  
    return maxi
```

Q : Pourquoi initialise-t-on `maxi = liste[0]` plutôt que `maxi = 0` ?

- ☐ C'est la même chose, 0 est suffisant
- ☐ Parce que si tous les éléments sont négatifs, initialiser à 0 donnerait un résultat faux
- ☐ Pour respecter une convention Python
- ☐ Parce que `liste[0]` est toujours le plus grand élément

Q : Que retourne `maximum([3, 7, 2, 9, 1])` ?

Activité 4 — Écrire une fonction

Écris une fonction `moyenne(liste)` qui retourne la moyenne des éléments d'une liste de nombres, ou 0 si la liste est vide.

Q : Que retourne `moyenne([4, 8, 12])` ?

Q : Quelle expression Python calcule la somme de tous les éléments de liste ?

Aide IA

☐ **Aide IA — prompt à copier :**

Je suis étudiant en BTS SIO SLAM et je révise l'algorithmie Python.
J'ai du mal à écrire des fonctions qui parcourent une liste et retournent un résultat. Pose-moi des questions étape par étape pour m'aider à construire ma fonction moi-même, sans me donner le code directement.

Solution

Voir la solution (à la fin du cahier)

Auto-évaluation

Où en es-tu ? ☐ Compris ☐ À revoir ☐ Bloqué

Gestion d'erreurs et exceptions

Objectifs :

- Comprendre ce qu'est une exception en Python
- Utiliser try/except pour gérer les erreurs
- Lever une exception avec raise

Rappel de cours

Quand Python rencontre une erreur, il **lève une exception** — le programme s'arrête si l'exception n'est pas gérée.

```
# Sans gestion d'erreur : le programme plante
x = int("bonjour")  # ValueError: invalid literal for int()
```

Gérer une exception avec try/except

```
try:
    valeur = int(input("Entrez un nombre : "))
    print(f"Le double est : {valeur * 2}")
except ValueError:
    print("Ce n'est pas un nombre valide.")
except ZeroDivisionError:
    print("Division par zéro impossible.")
finally:
    print("Fin du programme.")  # toujours exécuté
```

- Le bloc try contient le code qui peut échouer.
- Le bloc except gère l'erreur si elle se produit.
- Le bloc finally est toujours exécuté, qu'il y ait eu une erreur ou non.

Lever une exception avec raise

```
def deposter(solde, montant):
    if montant <= 0:
        raise ValueError("Le montant doit être positif.")
    return solde + montant

try:
    nouveau_solde = deposter(100, -50)
except ValueError as e:
    print(f"Erreur : {e}")  # Erreur : Le montant doit être positif.
```

Activité 1 — Lire du code avec try/except

```
def diviser(a, b):  
    try:  
        resultat = a / b  
        return resultat  
    except ZeroDivisionError:  
        print("Impossible de diviser par zéro.")  
        return None  
print(diviser(10, 2))  
print(diviser(10, 0))
```

Q : Que va afficher ce programme (dans l'ordre) ?

- ☐ 5.0 puis une erreur ZeroDivisionError
- ☐ 5.0 puis None [incorrect — il manque le print de la fonction]
- ☐ 5.0 puis "Impossible de diviser par zéro." puis None
- ☐ Une erreur ZeroDivisionError dès le début

Q : Que retourne diviser(10, 0) ?

- ☐ 0
- ☐ None
- ☐ Une exception ZeroDivisionError
- ☐ False

Activité 2 — Identifier le bon except

```
liste = [10, 20, 30]  
try:  
    print(liste[5])  
except ValueError:  
    print("Valeur incorrecte.")  
except IndexError:  
    print("Index hors limites.")  
except TypeError:  
    print("Mauvais type.")
```

Q : Quel message va s'afficher ?

- ☐ Valeur incorrecte.
- ☐ Index hors limites.
- ☐ Mauvais type.
- ☐ Aucun message, le programme plante

Q : Quelle exception Python lève-t-on en accédant à un index qui n'existe pas dans une liste ?

- ☐ ValueError
- ☐ KeyError
- ☐ IndexError
- ☐ TypeError

Activité 3 — Compléter un try/except

```
def lire_entier(texte):  
    """  
    Convertit texte en entier.  
    Retourne l'entier si possible, None sinon.  
    """  
    try:  
        return int(texte)  
    except _____:  
        return None  
print(lire_entier("42"))          # 42  
print(lire_entier("bonjour"))    # None
```

Q : Quelle exception faut-il mettre à la place des ____ pour que la fonction fonctionne correctement ?

Activité 4 — raise

```
def calculer_racine(n):  
    if n < 0:  
        raise ValueError("Impossible de calculer la racine d'un nombre négatif.")
```

```
    return n ** 0.5
try:
    print(calculer_racine(9))
    print(calculer_racine(-4))
except ValueError as e:
    print(f"Erreur : {e}")
```

Q : Que va afficher ce programme ?

- ☐ 3.0 puis une erreur non gérée
- ☐ {'3.0 puis "Erreur": 'Impossible de calculer la racine d'un nombre négatif.' '}
- ☐ Une erreur immédiate car $-4 < 0$
- ☐ 3.0 puis None

Aide IA

☐ **Aide IA — prompt à copier :**

Je suis étudiant en BTS SIO SLAM et j'apprends la gestion des exceptions en Python. Je ne comprends pas toujours quelle exception lever ni où mettre le try/except. Aide-moi à comprendre la logique générale avec un exemple simple, sans me donner directement le code de mon exercice.

Solution

Voir les solutions (à la fin du cahier)

Auto-évaluation

Où en es-tu ? ☐ Compris ☐ À revoir ☐ Bloqué

Git — commit, branche, merge

Objectifs :

- Comprendre le rôle de Git dans un projet de développement
- Lire et interpréter les commandes git add, commit, push
- Comprendre ce qu'est une branche et comment fonctionne le merge

Rappel de cours

Git est un outil de **contrôle de version** : il enregistre l'historique de tes modifications et permet de travailler à plusieurs sur le même code.

Workflow de base

```
# 1. Vérifier l'état du dépôt
git status
# 2. Ajouter des fichiers à l'index (staging)
git add mon_fichier.py
git add .          # ajoute tous les fichiers modifiés
# 3. Créer un commit (enregistrer les modifications)
git commit -m "feat: ajouter la classe Produit"
# 4. Envoyer sur le dépôt distant (GitHub, GitLab...)
git push
# Voir l'historique
git log --oneline
```

Branches

Une **branche** permet de travailler sur une fonctionnalité sans toucher au code principal (main).

```
# Créer et basculer sur une nouvelle branche
git checkout -b ma-fonctionnalite
# Revenir sur main
git checkout main
# Fusionner la branche dans main
git merge ma-fonctionnalite
# Lister les branches
git branch
```

Convention de messages de commit

Un bon message de commit décrit **ce qui a changé** et **pourquoi** :

Préfixe Signification

feat:	Nouvelle fonctionnalité
fix:	Correction de bug
refactor:	Réorganisation du code
docs:	Documentation
test:	Ajout ou modification de tests

Activité 1 — Lire un historique Git

```
$ git log --oneline
a3f9c12 fix: corriger le calcul de la TVA
e7b2d45 feat: ajouter la méthode appliquer_remise
c1d4e78 feat: créer la classe Produit avec __init__
9b3a2f1 init: initialiser le projet
```

Q : Quel est le commit le plus récent dans cet historique ?

- ☐ {'init': 'initialiser le projet'}
- ☐ {'feat': 'créer la classe Produit avec __init__'}
- ☐ {'fix': 'corriger le calcul de la TVA'}
- ☐ {'feat': 'ajouter la méthode appliquer_remise'}

Q : Un développeur corrige un bug d'affichage dans la vue. Quel préfixe de message de commit utiliser ?

- ☐ feat (nouvelle fonctionnalité)
- ☐ fix (correction de bug)
- ☐ refactor (réorganisation)
- ☐ docs (documentation)

Activité 2 — Ordre des commandes

Tu viens de modifier le fichier `produit.py` et tu veux enregistrer tes modifications.

Q : Dans quel ordre faut-il exécuter ces commandes ? A. `git commit -m "feat: ajouter methode afficher"` B. `git add produit.py` C. `git push`

- ☐ A, B, C

- ☐ B, A, C
- ☐ C, B, A
- ☐ B, C, A

Q : Quelle commande permet de voir quels fichiers ont été modifiés mais pas encore ajoutés à l'index ?

Activité 3 — Branches et merge

```
git checkout main
git checkout -b feature/login
# ... travail sur la fonctionnalité login ...
git add .
git commit -m "feat: ajouter le formulaire de login"
git checkout main
git merge feature/login
```

Q : Après ces commandes, que contient la branche main ?

- ☐ Uniquement le code de feature/login
- ☐ Le code original de main, sans les modifications de feature/login
- ☐ Le code de main fusionné avec les modifications de feature/login
- ☐ Une erreur, car on ne peut pas merger une branche feature

Q : Pourquoi travailler sur une branche séparée plutôt que directement sur main ?

- ☐ Parce que Git l'oblige toujours
- ☐ Pour isoler le développement d'une fonctionnalité et ne pas casser le code stable de main
- ☐ Pour aller plus vite
- ☐ Parce que les commits sur main sont impossibles

Aide IA

📄 Aide IA — prompt à copier :

```
Je suis étudiant en BTS SIO SLAM et j'apprends Git. Je comprends les commandes de base (add, commit, push) mais je ne suis pas sûr de comprendre à quoi servent vraiment les branches et le merge. Explique-moi avec un exemple concret de projet, pas à pas, sans me donner les réponses de mes exercices directement.
```

Solution

Voir les solutions (à la fin du cahier)

Auto-évaluation

Où en es-tu ? ☐ Compris ☐ À revoir ☐ Bloqué

Mini-projet CRUD — Gestion de contacts

Objectifs :

- Comprendre l'architecture modèle-DAO-contrôleur
- Lire et compléter un CRUD complet en Python
- Relier les notions de POO, SQL et gestion d'erreurs dans un projet réel

Présentation du projet

Tu vas travailler sur une application de **gestion de contacts** : une liste de personnes avec nom, prénom et email. L'application suit une architecture en couches :

- **Modèle** : la classe `Contact` représente un contact.
- **DAO** (Data Access Object) : la classe `ContactDAO` gère les accès à la base de données.
- **Contrôleur** : les fonctions qui relient la logique métier à l'interface.
- **Vue** : l'affichage en console.

C'est une version simplifiée de ce que vous ferez en 2ème année avec des frameworks web.

Couche 1 — Le modèle

```
class Contact:
    """Représente un contact dans le carnet d'adresses."""
    def __init__(self, nom, prenom, email, id=None):
        self.id = id
        self.nom = nom
        self.prenom = prenom
        self.email = email
    def __str__(self):
        return f"[{self.id}] {self.prenom} {self.nom} - {self.email}"
```

Q : Pourquoi `id` a-t-il une valeur par défaut de `None` dans `__init__` ?

- ☐ Par convention Python, tous les arguments doivent avoir une valeur par défaut
- ☐ Parce qu'un nouveau contact n'a pas encore d'id — il sera assigné par la base de données lors de l'insertion
- ☐ Parce que l'id n'est pas utile dans le modèle
- ☐ Pour éviter une erreur si on oublie de passer l'id

Couche 2 — Le DAO

```
import sqlite3
class ContactDAO:
    def __init__(self, chemin_db="contacts.db"):
        self.chemin_db = chemin_db
        self._initialiser()
    def _initialiser(self):
        """Crée la table contacts si elle n'existe pas."""
        with sqlite3.connect(self.chemin_db) as conn:
            conn.execute("""
                CREATE TABLE IF NOT EXISTS contacts (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    nom TEXT NOT NULL,
                    prenom TEXT NOT NULL,
                    email TEXT NOT NULL
                )
            """)
    def creer(self, contact):
        """Insère un contact et retourne son id."""
        with sqlite3.connect(self.chemin_db) as conn:
            curseur = conn.execute(
                "INSERT INTO contacts (nom, prenom, email) VALUES (?, ?, ?)",
                (contact.nom, contact.prenom, contact.email)
            )
            contact.id = curseur.lastrowid
            return contact
    def lire_tous(self):
        """Retourne la liste de tous les contacts."""
        with sqlite3.connect(self.chemin_db) as conn:
            rows = conn.execute(
                "SELECT id, nom, prenom, email FROM contacts ORDER BY nom"
            ).fetchall()
            return [Contact(r[1], r[2], r[3], r[0]) for r in rows]
    def supprimer(self, id_contact):
        """Supprime le contact avec l'id donné."""
        with sqlite3.connect(self.chemin_db) as conn:
            conn.execute("DELETE FROM contacts WHERE id = ?", (id_contact,))
```

Q : Dans la méthode `creer()`, pourquoi utilise-t-on des `"?"` dans la requête SQL plutôt que de concaténer les valeurs directement dans la chaîne ?

- ☐ C'est une convention de style uniquement
- ☐ Pour éviter les injections SQL — les `"?"` sont remplacés de façon sécurisée par SQLite
- ☐ Parce que Python ne permet pas les f-strings dans `sqlite3`
- ☐ Pour aller plus vite

Q : Que fait la ligne : `return [Contact(r[1], r[2], r[3], r[0]) for r in rows]`

- ☐ Elle retourne une liste de tuples bruts de la base de données
- ☐ Elle crée un objet `Contact` pour chaque ligne du résultat, et retourne la liste
- ☐ Elle filtre les contacts selon leur id
- ☐ Elle supprime les contacts en double

Couche 3 — Contrôleur et vue

```
def afficher_contacts(dao):
    """Affiche tous les contacts."""
    contacts = dao.lire_tous()
    if not contacts:
        print("Aucun contact enregistré.")
        return
    for c in contacts:
        print(c)

def ajouter_contact(dao):
    """Demande les infos et crée un contact."""
    print("\n--- Ajouter un contact ---")
    nom = input("Nom : ").strip()
    prenom = input("Prénom : ").strip()
    email = input("Email : ").strip()
    try:
        if not nom or not prenom or not email:
            raise ValueError("Tous les champs sont obligatoires.")
        contact = Contact(nom, prenom, email)
        dao.creer(contact)
        print(f"Contact ajouté avec l'id {contact.id}.")
    except ValueError as e:
        print(f"Erreur : {e}")

def menu_principal():
    """Boucle principale de l'application."""
    dao = ContactDAO()
    while True:
        print("\n=== Carnet de contacts ===")
        print("1. Afficher tous les contacts")
        print("2. Ajouter un contact")
        print("3. Supprimer un contact")
        print("0. Quitter")
        choix = input("Votre choix : ")
        if choix == "1":
            afficher_contacts(dao)
        elif choix == "2":
            ajouter_contact(dao)
        elif choix == "3":
            supprimer_contact(dao)
        elif choix == "0":
            print("Au revoir !")
```

```
        break
    else:
        print("Choix invalide.")
```

Q : Dans `ajouter_contact()`, pourquoi vérifie-t-on "if not nom or not prenom or not email" avant de créer le contact ?

- ☐ Pour respecter une règle Python sur les chaînes vides
- ☐ Pour s'assurer que l'utilisateur n'a pas laissé un champ vide, ce qui serait une donnée invalide
- ☐ Parce que SQLite refuse les chaînes vides automatiquement
- ☐ C'est inutile, SQLite gèrerait l'erreur tout seul

Activité — Compléter le CRUD

La méthode `supprimer_contact()` est manquante dans le contrôleur. Elle doit : 1. Demander l'id du contact à supprimer 2. Appeler `dao.supprimer(id)` avec gestion d'erreur si l'id n'est pas un entier

Q : Quelle exception Python faut-il attraper si l'utilisateur entre "abc" au lieu d'un nombre entier ?

Q : Voici une ébauche de `supprimer_contact` : `id_str = input("Id du contact : ")`
`try: id_contact = int(id_str) dao.supprimer(id_contact) print("Contact supprimé.")`
`except ValueError: print("Id invalide.")` Cette implémentation est-elle correcte et suffisante ?

- ☐ Non, il faut d'abord vérifier que le contact existe avant de le supprimer
- ☐ Oui, elle gère correctement le cas d'un id non numérique et appelle bien le DAO
- ☐ Non, on ne peut pas appeler `dao.supprimer()` directement
- ☐ Non, il faut utiliser un `while True` pour redemander l'id en cas d'erreur

Aide IA

☐ Aide IA — prompt à copier :

```
Je suis étudiant en BTS SIO SLAM et je travaille sur un mini-projet CRUD en Python avec SQLite. Je dois écrire la fonction supprimer_contact() qui demande un id à l'utilisateur et appelle le DAO. Je bloque sur [explique ton blocage]. Guide-moi pas à pas sans me donner le code complet directement.
```

Solution

Voir la solution complète (à la fin du cahier)

Auto-évaluation

Où en es-tu ? ☐ Compris ☐ À revoir ☐ Bloqué

Bilan final — Info-SLAM

Bravo, tu as terminé le cahier !

Ce cahier t'a fait travailler sur 8 thèmes essentiels du développement. Prends quelques minutes pour faire le point sur ce que tu as consolidé.

Ce que tu as travaillé

Activité	Thème	Je maîtrise
01	SQL — SELECT, WHERE, ORDER BY	
02	SQL — JOIN, GROUP BY, agrégats	
03	POO — classe, attribut, méthode	
04	POO — héritage et polymorphisme	
05	Algorithmie — boucles, conditions, fonctions	
06	Gestion d'erreurs et exceptions	
07	Git — commit, branche, merge	
08	Mini-projet CRUD complet	

Coche mentalement les thèmes que tu sens bien maîtriser. Ceux que tu laisses vides méritent une révision avant la rentrée.

Points clés à retenir

SQL : l'ordre des clauses ne change jamais — SELECT ... FROM ... WHERE ... GROUP BY ... ORDER BY Les JOIN permettent de croiser des tables liées par une clé étrangère.

POO : une classe est un moule, un objet est une instance. `self` désigne toujours l'objet courant. L'héritage permet de réutiliser du code ; le polymorphisme permet d'appeler la même méthode sur des objets différents.

Algorithmie : les fonctions doivent faire une seule chose et la faire bien. Penser aux cas limites (liste vide, valeur négative, etc.).

Exceptions : attraper l'exception la plus précise possible. Ne jamais masquer une erreur sans la traiter.

Git : add → commit → push. Toujours écrire des messages de commit clairs. Travailler sur des branches pour ne pas casser main.

Ton bilan personnel

Écris ici ce qui t'a semblé le plus utile, ce sur quoi tu veux encore travailler, et ce que tu as envie d'approfondir en 2ème année.

Mon bilan personnel :

Aide IA — pour continuer à progresser

📄 **Aide IA — prompt à copier :**

Je viens de terminer un cahier de révision BTS SIO SLAM couvrant SQL, la POO Python, l'algorithmie, les exceptions et Git.
Je veux identifier mes lacunes pour les travailler avant la rentrée.
Pose-moi 3 questions sur chacun de ces thèmes pour m'aider à évaluer mon niveau, sans me donner les réponses directement.

Pour la rentrée

Ces compétences sont directement utiles en 2ème année : - **SQL** : vous irez vers les requêtes imbriquées, les transactions, l'optimisation. - **POO** : vous utiliserez des frameworks (Symfony, Spring, Django...) qui s'appuient entièrement sur ces concepts. - **Git** : vous travaillerez en équipe avec des branches, des pull requests et des workflows de collaboration. - **CRUD** : c'est la base de toute application web ou mobile que vous développerez.

Bonne rentrée !